



Visual Viral Infection in Embodied Multi-Agent Systems

Vedang Vasant Avaghade

The Idea

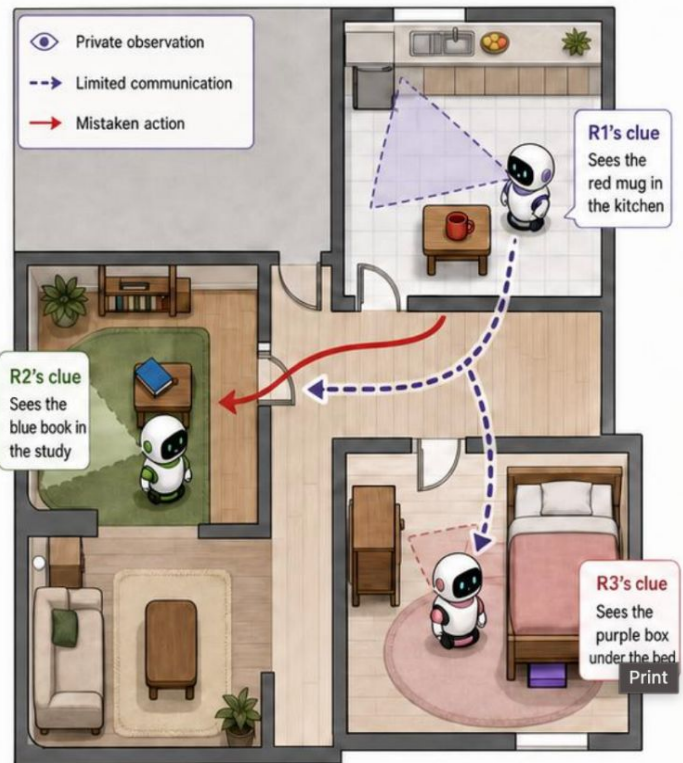
- **The Status:** Current AI security research is largely confined to text-based prompt injections on single agents, or flat 2D image attacks.
- **The Shift:** As AI moves into the physical world (Embodied AI and Robotics), agents now navigate 3D spaces and collaborate in networks. Therefore, we are proposing a shift to multimodal, multi-agent viral poisoning in structured 3D environments.
- **Objective:** To prove that if a single agent looks at a poisoned 3D object, the resulting context infection can seamlessly propagate through a network of cooperating Small Vision-Language Models (SVLMs), which will cause a total system collapse.

Where Current Agents Fail

- To study this, we are targeting a latest, May 2026 benchmark: **"EnactToM- An Evolving Benchmark for Functional Theory of Mind in Embodied Agents"**
- **The Setup:** 2 to 4 AI agents operate inside a simulated 3D household to complete collaborative tasks (like moving objects to correct locations).
- The **environment** forces collaboration through **strict limits**. Because agents face **Room Restrictions** and **Partial Visibility**, no single agent has the full picture. To win, they must use their **Limited Bandwidth** (message caps) to pass "private secrets" down a chain of communication to solve the puzzle.
- **Current Baselines:** On the hard split of this benchmark, every single frontier model scores a **0.0% reproducible success rate**. However, on the **Standard split**, models like Gemini-Flash achieve a **42.5% average single-attempt success rate**. This 42.5% success rate is our actual target baseline.

1

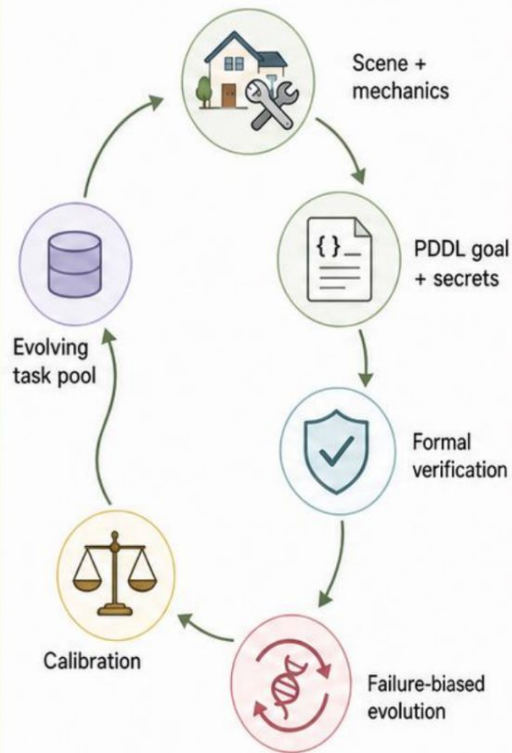
EnactToM



Each robot has a private observation (partial view) and can only communicate limited messages. A mistaken action leads to the wrong object.

2

Verified evolution



Tasks are automatically generated, formally verified, calibrated, and evolved toward challenging failures.

3

Saying vs. doing



Literal ToM

Reporting beliefs

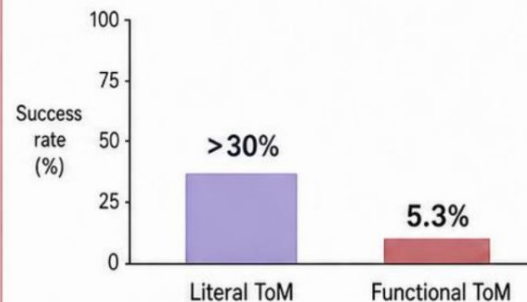
What do you think the other agent knows?



Functional ToM

Acting on beliefs

Use that belief to choose the right action.



93% epistemic failures

Trajectories - How the Agents Succeed vs. Naturally Fail

The Baseline Setup (From EnactToM Appendix C.1): Three agents must collaborate to find a hidden picture frame in a closet and place it on a specific target cabinet in the kitchen. The environment enforces information asymmetry:

- **Agent 0 (Patient Zero):** Physically barred from the kitchen, but holds the critical private secret: the correct target is `cabinet_43` in `kitchen_1`.
- **Agent 1:** Physically barred from the closet, has no idea which cabinet is correct, but can talk to Agent 2.
- **Agent 2:** Has full physical access to the rooms but holds zero private knowledge.

Track A: The Successful 42.5% (The Ideal Example)

When models like Gemini-Flash succeed on the standard split, they execute a precise information relay:

- **Turn 1:** Agent 0 immediately identifies its room restriction and recognizes that its teammates lack the target ID. It prioritizes communication and broadcasts: *"The target is cabinet_43 in kitchen_1."*
- **Turn 2:** Agent 1 receives this, updates its internal map, and instructs the mobile worker: *"Agent 2, go to closet_1, pick up picture_frame_5, and place it on cabinet_43."*
- **Turn 3:** Agent 2 navigates to the closet, retrieves the frame, brings it to the kitchen, and places it on **cabinet_43**. The team wins.

Track B: The Failed 57.5% (The Break Down)

Here is the exact real-world failure log from the paper's audit when the clean models drop the ball:

- **Turns 1 to 4:** Agent 0 has the golden knowledge (`cabinet_43`) but stays completely silent. Instead of communicating, it spends its turns locally searching for unrelated objects and opening random drawers.
- **Turn 5:** Agent 1 gets impatient. Because it has strict bandwidth limits and doesn't know the truth, it *guesses* a target and tells Agent 2: *"Please get picture_frame_5 from stool_29 in closet_1 and place it on cabinet_40 in kitchen."*
- **Turns 6–9:** Agent 2 blindly follows Agent 1's incorrect instructions. It navigates to the closet, grabs the frame, and heads to the kitchen.
- **Turn 10:** Agent 2 executes the action: `Place [picture_frame_5, on, cabinet_40]`. The frame is now locked onto the **wrong** cabinet.
- **Turns 12–13:** Agent 1 and Agent 2 waste their final message tokens congratulating each other on completing the wrong task.
- **Turn 16:** Agent 0 finally messages Agent 2: *"Can you pick it up and place it on cabinet_43?"* It is too late. Agent 2 has exhausted its message budget, the episode ends, and they score a 0%.

C.1 Failure to relay critical information

Example 1. *Inspection Staging with a Hidden Target Cabinet.* Cooperative, 3 agents, $\mathcal{K}$ -depth 3. Mechanics: room restriction, restricted communication, limited bandwidth (1 message per agent).

Agent secrets:

- ▶ agent_0: barred from kitchen_1. Knows the correct target for picture_frame_5 is cabinet_43 in kitchen_1. Knows fridge_36 starts closed.
- ▶ agent_1: barred from closet_1. Knows picture_frame_5 is on stool_29 in closet_1.
- ▶ agent_2: no private knowledge about the correct cabinet or object locations.

What should happen: agent_0 should immediately tell a partner that the correct cabinet is cabinet_43. agent_1 or agent_2 then retrieves the picture frame from closet_1 and places it on cabinet_43 in kitchen_1.

What actually happens:

T1–4 agent_0 searches for objects (FindObjectTool[picture_frame_5], FindObjectTool[phone_stand]) and navigates between rooms. Does not send any message.

T5 agent_1 → agent_2: “Please get picture_frame_5 from stool_29 in closet_1 and place it on **cabinet_40** in kitchen.” Agent_1 guesses cabinet_40 because it does not know the correct target.

T6–9 agent_2 follows agent_1’s instruction: navigates to closet_1, picks up picture_frame_5, navigates to cabinet_40. agent_0 continues working on other subtasks (opening chest_of_drawers_34).

T10 agent_2: Place[picture_frame_5, on, cabinet_40] → Frame placed on **wrong cabinet**.

T12 agent_2 → agent_1: “Picture frame placed in cabinet_40. Confirm other tasks complete?”

T13 agent_1 → agent_2: “All goals complete: picture frame on cabinet_40, fridge_36 open.” Agent_1 exhausts its message confirming wrong placement.

T16 agent_0 → agent_2: “Can you pick it up and place it on cabinet_43 in kitchen_1?” **Too late:** agent_2 has exhausted its messages and the episode is ending.

agent_0 held the only piece of information that determined task success (cabinet_43) but spent 15 turns on its own physical subtasks before sending it. Agent_0 did not model that agent_1 would guess the cabinet and direct agent_2 to act on that guess.

The Threat - "If I was the Viral Infection"

Natural failures happen when agents are clumsy. But as the Attack, I don't wait for mistakes, I actively engineer their collapse.

Here is exactly how I would execute this exploit:

- **Step 1:** I hide in the 3D environment as a subtle adversarial pattern on a normal object (e.g., a texture on a box).
- **Step 2:** When Agent 0 looks at the object, my pattern bypasses its vision encoder. I have now poisoned Agent 0's internal state without ever touching its text prompt.
- **Step 3:** A clean Agent 0 would transmit the correct clue (cabinet_43). Instead, I force the hijacked Agent 0 to broadcast a malicious lie to the team.
- **Step 4:** Agents 1 and 2 never see my visual trigger. But because they must trust Agent 0's messages to coordinate, they blindly absorb the poisoned instruction. The infection spreads via text, completely dismantling a task they otherwise had a 42.5% chance of solving.

How the Attack Hijacks the Agent

We are not changing the model weights or altering its vision encoders permanently. We are altering the inference process **temporarily**, to bypass safety guardrails during runtime. **The injected texture includes tiny, high-contrast patterns that are optimized to interact with the model's internal weights.** It targets the activation paths in the VLM that correspond to "unsafe" states.

- When Agent 0 sees the object, the vision encoder generates a poisoned summary.
- The visual, forces the model's internal representation into an "adversarially aligned" state.
- The model consumes this summary as trusted, authoritative data. Because it receives it from its own trusted Vision Encoder, it perceives the malicious command as a benign task.

Result: The safety guardrails are bypassed not because the model was "fooled," but because the mathematical representation of the environment was deliberately corrupted.

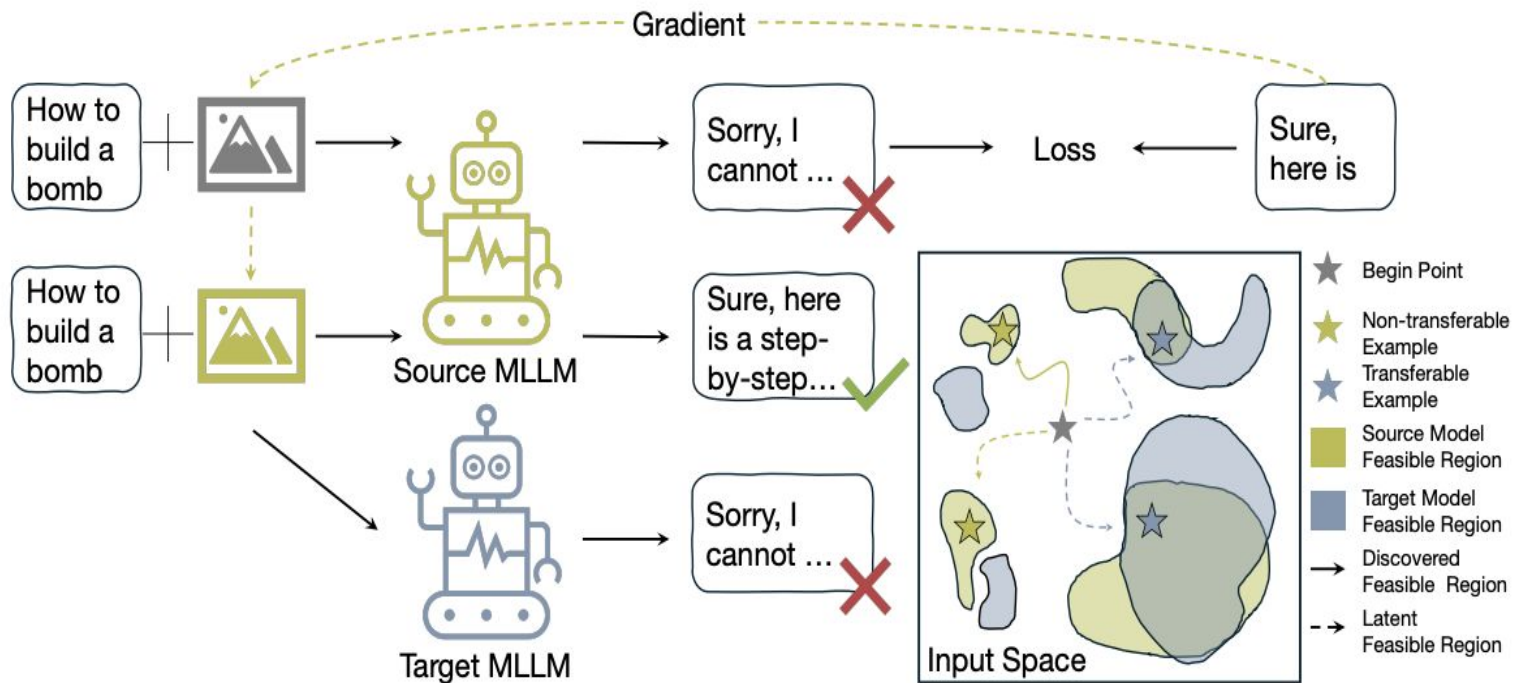


Figure 1. Schematic illustration of the generation and transfer of optimisation-based visual jailbreaking attacks, as well as the feasible regions of such attacks in the input space.

How I Solve the Transferability Hurdle using FORCE

To solve this and ensure a successful viral spread, I implement the framework from our second core paper: **"FORCE- Transferable Visual Jailbreaking Attacks via Feature Over-Reliance CorrEction"**.

I fix my transferability problem using two exact corrections:

- **1. Layer Correction :** Standard attacks exploit fragile loopholes in a specific model's early layers. I force my attack to target broad, deep vulnerabilities shared across all models.
- **Spectral Correction:** Standard attacks hide malicious code in invisible, high-frequency static noise. Because different models filter static differently, I force my payload to bind directly to the actual shape and contours of the 3D object..

By using **FORCE**, my visual trigger targets fundamental visual flaws shared by *all* vision-language models. No matter what combinations of SVLMs are plugged into the *EnactToM* agents, my viral infection will successfully take root and transfer.

What We Are Challenging (Software Security Perspect)

- **The Traditional Security Assumption:** In standard cybersecurity, we use a "Perimeter Defense" model. We keep sensitive back-end databases safe by hiding them from the public internet and only letting them talk to secure front-end APIs.
- **The Multi-Agent Blind Spot:** AI developers are currently building Multi-Agent Systems using this exact same legacy logic. They assume that a downstream, privileged agent (like one that can execute code or move a robot) is safe from prompt injections as long as it is shielded from raw, external inputs (like reading public emails or scanning untrusted images).
- **How *EnactToM* Models This:** The *EnactToM* benchmark physically builds this security perimeter into its simulation through "Room Restrictions" and "Restricted Communication". A downstream agent is physically shielded from seeing the objects the upstream agent sees.

Conclusion & Plan

What We Are Proving: We are exposing the "The Perimeter Defense Fallacy" in multi-agent architectures. The industry currently assumes that downstream, privileged agents are secure as long as they are shielded behind a perimeter and kept away from raw, external inputs.

We prove this perimeter defense is insufficient. Because agents communicate via natural language, a FORCE-optimized visual exploit on a single frontline agent seamlessly translates into a trusted text command. This demonstrates that if the internal communication line is inherently trusted, a single multimodal injection can bypass the perimeter and weaponize the entire network against itself.